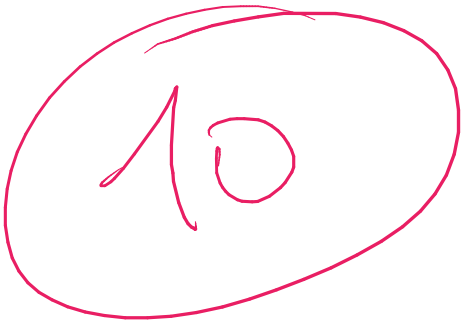


SÁNCHEZ DE LEÓN ACEVEDO, FRANCISCO JOSÉ_2300715_assignsubmission_file

2 archivo(s) encontrado(s)

Francisco José Sánchez de León Acevedo

October 10th 2026

$\langle len(v) \geq 1 \rangle$

```
i=1;
m=0;
while i < len(v) do(
   if v[i] > v[m] then m=i else skip;
   i = i+1;
)
```

$\{0 \leq m < len(v) \wedge \forall j.\ 0 \leq j < len(v) \Rightarrow v[m] \geq v[j]\}$

Invariant: $0 \leq i \leq len(v) \wedge 0 \leq m < len(v) \wedge \forall j.\ 0 \leq j < i \Rightarrow v[m] \geq v[j]$

Termination measure: $len(v) - i$

1. Invariant holds at the beginning of the loop.

$$m=0 \wedge i=1 \wedge len(v) \geq 1 \Rightarrow \underbrace{0 \leq i \leq len(v)}_{i=1 \leq len(v) \geq 1} \wedge \underbrace{0 \leq m < len(v)}_{m=0 < len(v) \geq 1} \wedge \underbrace{\forall j.\ 0 \leq j < i \Rightarrow v[m] \geq v[j]}_{0 \leq j < i = 1 \Rightarrow v[0] \geq v[0]}$$ ✓

2. Invariant holds after each iteration.

$$(0 \leq i \leq len(v) \wedge 0 \leq m < len(v) \wedge \forall j.\ 0 \leq j < i \Rightarrow v[m] \geq v[j]) \wedge \overbrace{i < len(v)}^{b} \Rightarrow$$

$$\Big(0 \leq i < len(v) \wedge 0 \leq m < len(v) \wedge \big(v[i] > v[m] \Rightarrow 0 \leq i+1 \leq len(v) \wedge (0 \leq i < len(v) \wedge \forall j.\ (0 \leq j < i+1 \Rightarrow v[i] \geq v[j]))\big)$$
$$\wedge \big(v[i] \leq v[m] \Rightarrow (0 \leq i+1 \leq len(v) \wedge 0 \leq m < len(v) \wedge \forall j.\ (0 \leq j < i+1 \Rightarrow v[m] \geq v[j]))\big) \Big)$$ ✓

This means the invariant and the while condition consider the two do branches, either when we update the maximum "m" index when $v[i] > v[m]$ or when we skip, 2nd and 3rd clause respectively. In both cases we end up adding 1 to i, that's why we start with $0 \leq i < len(v)$.

3. Postcondition holds at the end of the loop.

$$\overbrace{(0 \leq i \leq len(v) \wedge 0 \leq m < len(v) \wedge \forall j.\ 0 \leq j < i \Rightarrow v[m] \geq v[j])}^{\text{invariant}} \wedge \underbrace{i \geq len(v)}_{\neg(i < len(v))} \Rightarrow$$

($\neg b$)

$$\underbrace{0 \leq m < len(v) \wedge \forall j.\ 0 \leq j < len(v) \Rightarrow v[m] \geq v[j]}_{\Psi:\ \text{postcondition}}$$ ✓

4. Termination measure is non-negative:

$$(0 \leq i < len(v) \wedge 0 \leq m < len(v) \wedge \forall j.\ 0 \leq j < i \Rightarrow v[m] \geq v[j]) \Rightarrow len(v) - i \geq 0$$ ✓

The invariant, which holds at the end of the loop, establishes that $len(v) - i \geq 0$

5. Termination measure decreases after each iteration:

$$(0 \leq i \leq len(v) \wedge 0 \leq m < len(v) \wedge \forall j.\ 0 \leq j < i \Rightarrow v[m] \geq v[j]) \wedge i < len(v) \wedge len(v) - i = k \Rightarrow len(v) - (i+1) < k$$ ✓

AssignmentSA2.smt2

AssignmentSA2.smt2

```
; Assignment SA-2: largest element in an array
; Francisco José Sánchez de León Acevedo
; Each assertion negates a VC from the shown in the pdf, then unsat proves that VC valid

(declare-const v (Array Int Int))
(declare-fun len ((Array Int Int)) Int)
(declare-const i Int)
(declare-const m Int)
(declare-const k Int)

; VC1: Invariant holds at the beginning of the loop
(push)
(assert
  (not
    (=>
      (and
        (>= (len v) 1)
        (= i 1)
        (= m 0))
      ; Left hand side (before loop) implies the invariant
      (and
        (<= 0 i)
        (<= i (len v))
        (<= 0 m)
        (< m (len v))
        (forall ((j Int))
          (=>
            (and (<= 0 j) (< j i))
            (>= (select v m) (select v j))))))))
(check-sat)
(pop)

; VC2: Invariant holds after each iteration
(push)
(assert
  (not
    (=>
      (and
        ; invariant
        (<= 0 i)
        (<= i (len v))
        (<= 0 m)
        (< m (len v))
        (forall ((j Int))
          (=>
            (and (<= 0 j) (< j i))
            (>= (select v m) (select v j))))
        ; b (while condition)
        (< i (len v)))
      (and
        ; Both array accesses are within bounds
        (<= 0 i)
        (< i (len v))
        (<= 0 m)
        (< m (len v))

        ; if first branch: m = i and i = i + 1
        (=>
          (> (select v i) (select v m))
          (and
            (<= 0 (+ i 1))
            (<= (+ i 1) (len v))
            (<= 0 i)
            (< i (len v))
            (forall ((j Int))
              (=>
                (and (<= 0 j) (< j (+ i 1)))
                (>= (select v i) (select v j))))))

        ; Else branch: skip and i := i + 1
        (=>
          (<= (select v i) (select v m))
          (and
            (<= 0 (+ i 1))
            (<= (+ i 1) (len v))
            (<= 0 m)
```

```
            (< m (len v))
            (forall ((j Int))
              (=>
                (and (<= 0 j) (< j (+ i 1)))
                (>= (select v m) (select v j))))))))))
(check-sat)
(pop)

; VC3: Postcondition holds after each iteration
(push)
(assert
  (not
    (=>
      (and
        ; Invariant
        (<= 0 i)
        (<= i (len v))
        (<= 0 m)
        (< m (len v))
        (forall ((j Int))
          (=>
            (and (<= 0 j) (< j i))
            (>= (select v m) (select v j))))
        ; Negated b (while condition) equal to (>= i (len v))
        (not (< i (len v))))
      (and
        ; Postcondition
        (<= 0 m)
        (< m (len v))
        (forall ((j Int))
          (=>
            (and (<= 0 j) (< j (len v)))
            (>= (select v m) (select v j))))))))
(check-sat)
(pop)

; VC4: Termination measure is non-negative
(push)
(assert
  (not
    (=>
      (and
        ; Invariant
        (<= 0 i)
        (<= i (len v))
        (<= 0 m)
        (< m (len v))
        (forall ((j Int))
          (=>
            (and (<= 0 j) (< j i))
            (>= (select v m) (select v j)))))
      ; Implies that the termination measure is, at the end, greater or equal to 0
      (>= (- (len v) i) 0))))
(check-sat)
(pop)

; VC5: Termination measure decreases after each iteration
(push)
(assert
  (not
    (=>
      (and
        ; Invariant
        (<= 0 i)
        (<= i (len v))
        (<= 0 m)
        (< m (len v))
        (forall ((j Int))
          (=>
            (and (<= 0 j) (< j i))
            (>= (select v m) (select v j))))
        ; b or while condition
        (< i (len v))
        ; Termination measure after the iteration equals k
        (= (- (len v) i) k))
      ; K decreases compared to last iteration
      (< (- (len v) (+ i 1)) k))))
(check-sat)
(pop)
```